

Assignment 5: Simulating Orbital and Satellite Dynamics

1 Introduction

In this assignment the student will learn how to create simulators for satellites that combine orbital mechanics and attitude dynamics.

After completing this assignment the student will learn:

- How to implement scenarios that combine orbital mechanics and satellite attitude kinematics and dynamics.

2 Preliminaries

Before describing the steps needed to complete today's assignment, some fundamentals are needed. Specifically, the fundamentals of simulating systems on different time scales. This section is based on Lecture 3-6.

2.1 Simulation Time Steps

In Assignment 3 we discussed simulating dynamical systems as recursively solving an initial value problem

$$\dot{\mathbf{x}}(t) = \mathbf{f}(t, \mathbf{x}(t)), \quad \mathbf{x}(0) = \mathbf{x}_0$$

based on a Taylor series expansion truncated to order p

$$\mathbf{x}(t+h) = \sum_{n=0}^p \frac{1}{n!} \frac{d^n \mathbf{x}(t)}{dt^n} h^n$$

where an approximation of the solution at $t+h$ is obtained from an approximate at time t . While we looked at several different algorithms for doing so, they all have one thing in common and that is that they require selection of the time step h . If h is too large $\mathbf{x}(t)$ will likely diverge and if it is too small, it will take a long time to simulate and it could increase numerical errors. Therefore selecting a proper time step h becomes important and typically we want the largest time step that still ensures some level of required accuracy. The difficulty comes from the fact that this choice of h depends on the system we are simulating, because the larger the neglected higher order terms in the Taylor series are the smaller h needs to be. For instance forward Euler uses a first-order polynomial

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h \cdot \mathbf{f}(t, \mathbf{x}(t))$$

which assumes that over the interval t to $t+h$

$$\frac{1}{n!} \frac{d^n \mathbf{x}(t)}{dt^n} h^n \approx \mathbf{0}, \quad \forall n > 1$$

and if it doesn't hold then we need to make h smaller. Since this condition applies to the derivatives of $\mathbf{x}(t)$ we can informally say that fast changing systems (we'll just call them fast systems) require a smaller time step than slow systems.

2.1.1 Simulating systems with different time steps

When simulating the orbital dynamics and attitude dynamics together we quickly run into a time scaling issue. We want an accurate and efficient simulation but the time step for the orbital dynamics can typically be much larger than the time step for the attitude dynamics.

As a sidenote the orbital mechanics in Assignment 2 can be propagated with any sized time step without losing accuracy because at each time step we are just evaluating an analytical solution to the fundamental equation of relative two-body motion. With that said as we saw in Assignment 3, if we add anything to the orbital dynamics such as non-impulsive orbital maneuvers or orbital perturbations we then can't use the analytical solution anymore and have to numerically solve it.

There are several ways to deal with differences in time steps where the simplest method is to use the time step for the attitude dynamics to also propagate the orbital mechanics. The downside is that propagating the orbital mechanics at such a small time scale will eventually turn out to be very inefficient.

A more efficient variation is to still use the smallest time step for the attitude dynamics, but only update the orbital mechanics according to its own time step. The downside to this is that between the orbital dynamics time steps the satellite remains stationary such that any coupling between the orbital and attitude dynamics won't be properly accounted for. This can be especially problematic when simulating sensors such as magnetometers where its output depends on the satellite's position relative to the Earth.

A better solution is to divide the overall system we are simulating into smaller subsystems based on their required time step (i.e. fast and slow subsystems) and then use the largest time step as the main simulator time step. Then within each subsystem that require a smaller time step we subdivide the simulator time step into N substeps and run N updates on it. In simulating the overall system we update the subsystems from slowest to fastest. Whenever a faster subsystem requires the output of a slower subsystem at some substep we use interpolation (typically linear interpolation) to efficiently approximate the slower subsystem's output at the substep.

2.2 Propagating classical orbital parameters

Previously we have propagated the alternative/TLE parameters since the classical orbital parameters do not include time. However, there is a fairly simple way to propagate θ through the specific relative angular momentum h and r since

$$r = \frac{h^2}{\mu} \frac{1}{1 + e \cos \theta}$$

$$\dot{\theta} = \|\boldsymbol{\omega}_{io}^i\| = \frac{h}{r^2}$$

which therefore allows us to propagate θ similarly to Algorithm 5 in Assignment 2

$$\theta = \theta + \dot{\theta} \Delta t, \quad \text{If } \theta \geq 2\pi \rightarrow \theta = \theta - 2\pi$$

3 Orbital Mechanics and Attitude Dynamics

Assignment 5.1

1. Extend orbit.lib.py with a function

```
1 def orbit_frame_from_state(r_i, v_i): #returns  $q_{io}$ ,  
     $\omega_{io}^i$ ,  $\dot{\omega}_{io}^i$ 
```

2. Extend orbit.lib.py with the following classes:

```
1 class orbit_classic:  
2     def __init__(self, h, e, theta, O, i, w):  
3         self.h = h  
4         :  
5         :
```

```
1 class orbit_tle:  
2     def __init__(self, n, e, M_e, O, i, w):  
3         self.n = n  
4         :  
5         :
```

each class should also have the following methods

```
1 def propagate(self, t_step): #propagates orbit by  
     $t_{step}$   
2 def get_params(self): #returns orbital parameters  
3 def get_state(self): #returns  $r^i$ ,  $v^i$   
4 def get_orbit_frame(self): #returns  $q_{io}$ ,  $\omega_{io}^i$ ,  $\dot{\omega}_{io}^i$ 
```

Assignment 5.2

1. Extend the rigid-body class to also include translational kinematics and dynamics.
2. Extend sat.lib.py with a ADCS_PD class that implements the PD controller from Assignment 4:

```
1 class ADCS_PD:  
2     def __init__(self, k1, k2, f_c, J):  
3         self.k1 = k1  
4         self.k2 = k2  
5         self.J = J  
6         self.tau = np.zeros(3)  
7     def update(self, q_ib, w_b_ib, q_io, w_i_io): #  
        updates self.tau  
8         :  
9     def get_control(self):  
10        return self.tau
```

for now we assume that $q_{id} = q_{io}$, $\omega_{id}^i = \omega_{io}^i$ and $\dot{\omega}_{id}^i = \dot{\omega}_{io}^i$

Assignment 5.3

1. Modify the Satellite class to include the following:
 A rigid-body
 An orbit
 An ADCS
 N (substeps+1)
 let the constructor be:

```

1 def __init__(self, q_ib, w_b_ib, J, r=np.zeros(3), v=np
  .zeros(3), m=1, orbit=None, substeps=0):
2     self.orbit = orbit
3     if self.orbit is not None:
4         r, v = self.orbit.get_state()
5     self.body = RigidBody(r, v, m, q, w, J)
6     self.N = substeps+1
7     self.ADCS = ADCS_PD(1e-5, 2e-4, J)

```

Modify the update method to be

```

1 def update(self, t_k, t_step):
2     if self.orbit:
3         self.update_with_orbit(t_k, t_step)
4     else:
5         self.update_with_dynamics(t_k, t_step)

```

example implementations of update_with_orbit and update_with_dynamics is given in the appendix. Finally add the following convenience methods

```

1 def get_state(self):
2     return self.body.get_state()
3 def get_orbit_frame(self):
4     if self.orbit:
5         return self.orbit.get_orbit_frame()
6     else:
7         r, v, _, _ = self.body.get_state()
8         return ol.orbit_frame_from_state(r, v)

```

Note: Eventhough we have define two different orbit classes, there is no distinction between them in the Satellite class. Instead we are taking advantage of the 'duck typing' principle which python is built upon. As long as the thing self.orbit is set to in the constructor supports the functionality ('get_orbit_frame', 'get_state', 'propagate') required by the Satellite class it doesn't matter if it is a orbit_classic or orbit_tle. In other words if it walks and quacks like a duck, then we can treat it as if it is a duck.

Assignment 5.3

1. Create a scenario that simulates the following satellite over one orbit:

HINCUBE

1 39445U 13066AF 26114.13732855 .00006562 00000+0 43037-3 0 9997

2 39445 97.7929 81.8923 0034704 37.1228 323.2390 15.07529327671380

Use the simulation parameters

```
1 sim_config = {'t_0':0, 't_e':5731, 't_step':2, '
               speed_factor':1, 'anim_dt':1.0/25.0, '
               scale_factor':1000, 'visualise':True}
```

with initial conditions and parameters

$$\mathbf{q}_{ib}(0) = [1 \ 0 \ 0 \ 0]^\top$$

$$\boldsymbol{\omega}_{ib}^b(0) = [0 \ 0 \ 0]^\top$$

$$\mathbf{J}^b = \begin{bmatrix} 0.00146519 & 0.00001703 & -0.00000633 \\ 0.00001703 & 0.00151512 & -0.00001598 \\ -0.00000633 & -0.00001598 & 0.00146333 \end{bmatrix}$$

and substeps set to 50. Watch the satellite complete one full orbit. If you see anything strange report it to me on canvas.

A Example implementation of update methods

```

1 def update_with_orbit(self, t_k, t_step):
2     r_0, v_0 = self.orbit.get_state()
3     self.orbit.propagate(t_step)
4     r_1, v_1 = self.orbit.get_state()
5     t_sub = t_step/self.N
6     for n in range(0, self.N):
7         r_i = r_0 + n/self.N*(r_1-r_0)
8         v_i = v_0 + n/self.N*(v_1-v_0)
9         _, q_ib, w_b_ib = self.get_state()
10        q_io, w_i_io, _ = ol.orbit_frame_from_state(r_i, v_i)
11        self.ADCS.update(q_ib, w_b_ib, q_io, w_i_io)
12        tau_u = self.ADCS.get_control()
13        self.body.update(t_k, t_sub, np.zeros(3), tau_u)
14        t_k += t_sub
15    self.body.p, self.body.v = self.orbit.get_state()

1 def update_with_dynamics(self, t_k, t_step):
2     t_sub = t_step/self.N
3     for n in range(0, self.N):
4         r_i, v_i, q_ib, w_b_ib = self.get_state()
5         q_io, w_i_io, dw_i_io = self.get_orbit_frame()
6         self.ADCS.update(t_k, q_ib, w_b_ib, q_io, w_i_io, dw_i_io)
7         tau_u = self.ADCS.get_control()
8         f = -ol.mu/np.linalg.norm(r_i)**3*r_i
9         self.body.update(t_k, t_sub, f, tau_u)
10        t_k += t_sub

```